

1 FSDP Architecture

Fully Sharded Data Parallel (FSDP) lets us train models that are too large for one GPU [2]. In plain data parallelism, every GPU keeps a full copy of the model. FSDP does not. It splits the parameters, gradients, and optimizer states across all GPUs, so each GPU keeps only a small piece. When a GPU needs a layer, it collects the missing pieces from the other GPUs, runs the layer, and then throws the borrowed pieces away. This way, no single GPU ever has to hold the whole model at once.

1.1 Notation and Definitions

1 lists the symbols we use in this section.

Symbol	Meaning
N	Number of GPUs
Ψ	Total number of model parameters
Ψ_u	Number of parameters in one FSDP unit
U	Number of FSDP units in the model
W	Parameters of one unit (flattened)
W_i	The i -th shard of W , owned by rank i
$g^{(i)}$	Local gradient computed on GPU i
g_i	The i -th shard of the averaged gradient g
K	Optimizer-state bytes per parameter

Table 1: Notation used in this section.

Definition 1 (Sharding). Sharding splits a tensor into N pieces that do not overlap. Rank i owns and stores the i -th piece.

Definition 2 (FSDP unit). An FSDP unit is a group of layers that FSDP treats as one block. FSDP flattens, shards, gathers, and frees the parameters of a unit together.

1 shows how we split a six-layer model into FSDP units. Unit 1 holds Layers 1 and 2. Unit 2 holds Layers 4 and 5. FSDP flattens and shards each of these units on its own. Unit 0 wraps the whole model. It picks up every layer that no inner unit claims, which here means Layers 0 and 3.

The right side of the figure shows the life of one unit. In the forward pass, the unit gathers its full parameters, runs its layers, and frees the shards that other ranks own. In the backward pass, it gathers the parameters again, computes gradients, frees the borrowed shards once more, and syncs the gradients across ranks. At any moment, only the active unit holds its full parameters. So the way we wrap the model decides the peak memory. Smaller units mean less memory at the peak.

1.2 Model State and Sharding

1.2.1 Components of Model State

The model state has three parts: parameters, gradients, and optimizer states. The parameters are the weights we train. The gradients tell us how to change those weights, and we compute them in the backward pass. The optimizer states are extra values that the optimizer keeps between steps.

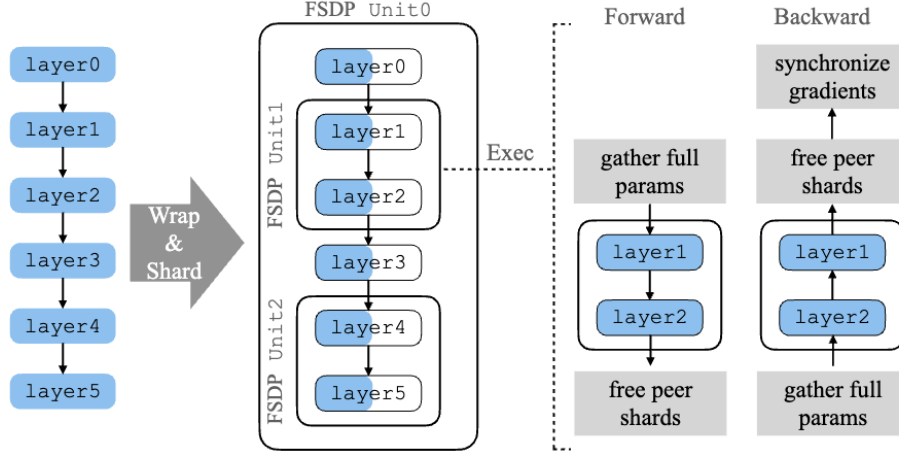


Figure 1: Wrapping a model into FSDP units. Layers 0 and 3 are not wrapped separately, so they belong to the outer unit, Unit 0.

Take Adam with mixed-precision training as an example. Here the optimizer keeps an fp32 copy of the weights, because fp16 alone loses too much precision during updates. It also keeps the first and second moment of the gradients, again in fp32. Together, these three values take K bytes per parameter [1].

1.2.2 Replication in DDP versus Sharding in FSDP

In Distributed Data Parallel (DDP), every GPU keeps a full copy of all Ψ parameters. Each GPU also keeps a full copy of the gradients and the optimizer states. All N GPUs store the same data. This wastes a lot of memory, and large models quickly run out of room.

FSDP removes this waste. It takes the flattened parameters W of each unit and splits them into N shards. Rank i stores and updates only its own shard W_i .

FSDP shards the gradients and the optimizer states in the same way. Rank i keeps only the gradient shard g_i and the optimizer state for the parameters in W_i . 2 compares the two layouts. In DDP each GPU holds everything. In FSDP each GPU holds one N -th of everything. This is why FSDP can train models that DDP cannot fit.

1.2.3 Flattening and Padding

A layer usually has several tensors, such as a weight matrix and a bias vector. Sharding each tensor one by one would be slow and messy. So FSDP first joins all parameters of a unit into one long 1-D tensor. We call this tensor the **FlatParameter**.

The length of the **FlatParameter** is Ψ_u . This number may not divide evenly by N . When it does not, FSDP adds a few zeros at the end (padding) until the length is a multiple of N . Then it cuts the tensor into N equal shards. Each shard has the size given by 1.

$$\text{shard size} = \left\lceil \frac{\Psi_u}{N} \right\rceil \quad (1)$$

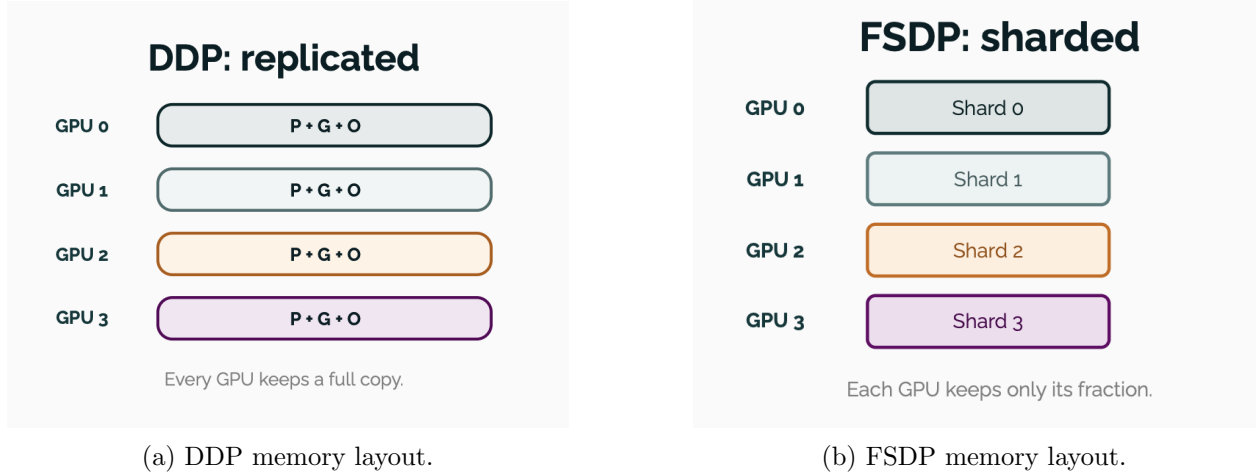


Figure 2: Model-state memory layout under DDP (every GPU holds a full replica) and FSDP (every GPU holds one shard).

3 shows an example with 16 GPUs. The unit has a 3×4 weight (12 values) and a bias with 3 values, so $\Psi_u = 15$. FSDP joins them into one tensor of length 15 and adds one padding value to reach 16. Each GPU then gets exactly one value. Ranks 0 to 14 hold real parameters, and rank 15 holds only the padding.

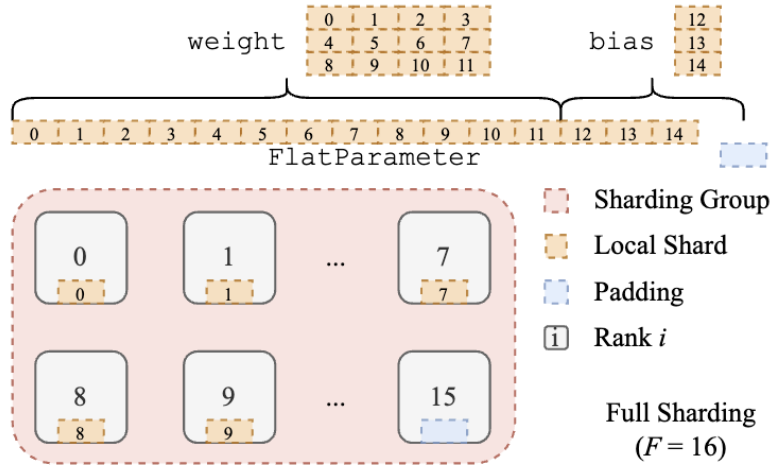


Figure 3: `FlatParameter` sharding across 16 GPUs. The weight and bias are joined into one 1-D tensor, padded to length 16, and split so that each rank owns one element. The symbol F in the figure is N in our notation.

1.3 Collective Communication Primitives

FSDP moves data between GPUs with two collective operations: All-Gather and Reduce-Scatter. In a collective, all N ranks take part at the same time.

1.3.1 All-Gather

All-Gather turns shards into a full tensor. Each rank starts with its own shard W_i . Each rank sends its shard to every other rank. At the end, every rank holds the full tensor W .

4 shows this for four GPUs. Before the call, each GPU stores only $|W|/N$ values. This shard is persistent: the GPU keeps it for the whole training run. After the call, each GPU holds all $|W|$ values. This full copy is transient: the GPU frees it as soon as it finishes using it. Each rank already has its own shard, so it only receives the other $N - 1$ shards. 2 gives the amount of data each rank receives.

$$V_{AG} = \frac{N-1}{N} |W| \quad (2)$$

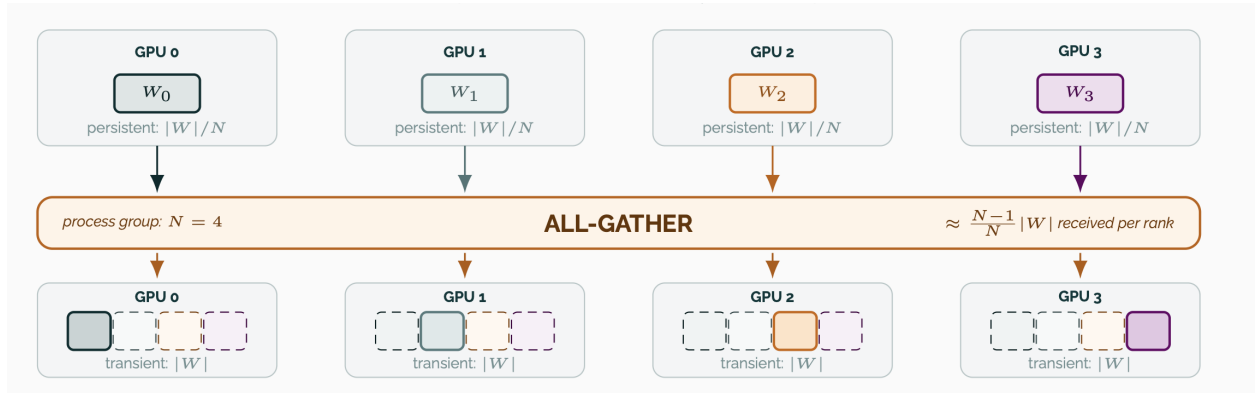


Figure 4: All-Gather on four GPUs. Each GPU starts with one persistent shard W_i and ends with a transient copy of the full tensor W .

1.3.2 Reduce-Scatter

Reduce-Scatter works in the other direction. Each rank starts with a full tensor G of the same size. The ranks add their tensors together element by element. But no rank ends up with the full sum. Instead, rank i gets only the i -th shard of the sum.

Reduce-Scatter is one single collective. It is not a Reduce call followed by a Scatter call. No rank ever builds the full sum, so no rank needs the extra memory for it. Each rank sends and receives about the same amount of data as in All-Gather:

$$V_{RS} = \frac{N-1}{N} |G| \quad (3)$$

1.3.3 Relation to All-Reduce

DDP syncs its gradients with All-Reduce. All-Reduce adds the tensors from all ranks and gives the full sum to every rank. We can split All-Reduce into two steps. First, a Reduce-Scatter gives each rank one shard of the sum. Then, an All-Gather sends every shard to every rank. So All-Reduce equals Reduce-Scatter plus All-Gather, and it moves about twice as much data as either one alone.

FSDP uses these two halves on their own. It uses All-Gather to rebuild parameters and Reduce-Scatter to sync gradients. Note that a plain All-Reduce gives a sum. FSDP wants the average gradient, so it also divides the result by N .

1.4 Forward Pass

1.4.1 Why a Layer Needs Full Parameters

Think of a linear layer that computes $y = Wx$. Every output value depends on a full row of W . Rank i holds only the shard W_i , which is a small slice of the flattened tensor. With only W_i , rank i cannot compute y . It needs all of W . So before a unit runs, every rank must collect the full parameters of that unit.

1.4.2 Gather, Compute, Reshard

The forward pass handles one unit at a time in three steps:

1. **Gather.** The ranks call All-Gather on their shards. Every rank now holds the full W of the unit.
2. **Compute.** Each rank runs the unit on its own mini-batch.
3. **Reshard.** Each rank frees the parts of W that it does not own. It keeps only W_i .

The full parameters live only for a short time, while the unit runs. The shard W_i stays in memory for the whole training run. Then the next unit repeats the same steps.

1.5 Backward Pass and Gradient Synchronization

1.5.1 Re-gathering Parameters

The backward pass also needs the full parameters, because computing the gradient of a layer uses its weights. But FSDP freed those weights right after the forward pass. So each unit calls All-Gather again before its backward step. The backward pass visits the units in reverse order, from the last unit to the first. After the backward step, each rank frees the borrowed parameters again.

1.5.2 Local Gradients

Each GPU trains on a different mini-batch. So when GPU i runs the backward pass, it gets its own local gradient $g^{(i)}$. These local gradients differ from GPU to GPU. If each GPU used its own gradient to update the model, the shards would drift apart. So the GPUs must first agree on one gradient.

1.5.3 Synchronization by Reduce-Scatter

FSDP uses the average of all local gradients as the true gradient. 4 defines it. Rank i does not need the whole average. It only updates W_i , so it only needs the matching shard g_i . A Reduce-Scatter gives each rank exactly that shard. After the call, FSDP divides by N to turn the sum into an average.

$$g = \frac{1}{N} \sum_{i=1}^N g^{(i)}, \quad \text{rank } i \text{ keeps only } g_i \quad (4)$$

2 and 5 show a small example with four GPUs. Each GPU computes a local gradient with four values. The average of the four vectors is [1.5, 1.5, 2.0, 1.5]. GPU i keeps only the i -th value of this average. For example, GPU 2 keeps $(0 + 4 + 1 + 3)/4 = 2.0$.

Table 2: Worked four-GPU example of gradient synchronization by Reduce-Scatter.

GPU	Local gradient $g^{(i)}$	Shard kept g_i
0	[2, 1, 0, 3]	1.5
1	[1, 2, 4, 0]	1.5
2	[3, 1, 1, 2]	2.0
3	[0, 2, 3, 1]	1.5
Averaged gradient $g = [1.5, 1.5, 2.0, 1.5]$		

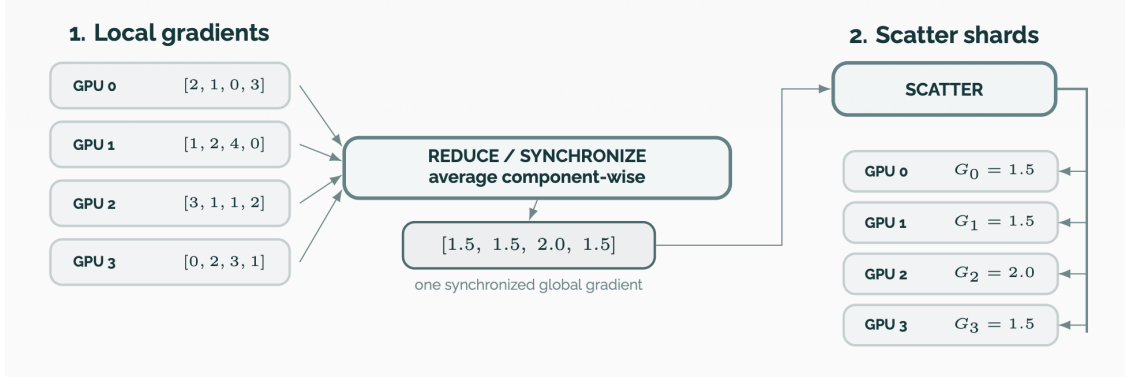


Figure 5: Gradient synchronization on four GPUs. The local gradients are averaged element by element, and each GPU keeps one shard of the result. The symbol G_i in the figure is g_i in our notation.

1.6 Optimizer Step

After the backward pass, rank i holds three things for its shard: the parameters W_i , the averaged gradient g_i , and the optimizer state for W_i . That is all it needs to run the optimizer. So each rank updates its own shard on its own. The ranks do not talk to each other in this step. Together, the N updated shards form the full updated model, which is the same model that DDP would produce.

1.7 The Complete Training Iteration

1 puts all the steps together. It shows one FSDP training iteration on rank i :

Algorithm 1 One FSDP training iteration on rank i

Require: Parameter shards $W_i^{(u)}$ and optimizer-state shards for every unit $u = 1, \dots, U$; local mini-batch $x^{(i)}$
 // Forward pass
1: **for** $u = 1$ **to** U **do**
2: $W^{(u)} \leftarrow \text{ALLGATHER}(W_i^{(u)})$
3: Compute forward of unit u with $W^{(u)}$
4: Free all of $W^{(u)}$ except $W_i^{(u)}$ $\triangleright$ reshard
5: **end for**
6: Compute loss $\mathcal{L}$
 // Backward pass
7: **for** $u = U$ **down to** 1 **do**
8: $W^{(u)} \leftarrow \text{ALLGATHER}(W_i^{(u)})$
9: Compute local gradient $g^{(i),u}$ of unit u
10: Free all of $W^{(u)}$ except $W_i^{(u)}$
11: $g_i^{(u)} \leftarrow \text{REDUCESCATTER}(g^{(i),u})/N$
12: Free $g^{(i),u}$
13: **end for**
 // Optimizer step (local, no communication)
14: **for** $u = 1$ **to** U **do**
15: Update $W_i^{(u)}$ using $g_i^{(u)}$ and its optimizer-state shard
16: **end for**

References

- [1] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. ZeRO: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.
- [2] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.